



REPÚBLICA BOLIVARIANA DE VENEZUELA  
UNIVERSIDAD CENTROCCIDENTAL "LISANDRO ALVARADO"  
DECANATO DE CIENCIA Y TECNOLOGÍA  
MAESTRÍA EN CIENCIAS DE LA COMPUTACIÓN  
BARQUISIMETO – ESTADO LARA

## **Implementación de Autómatas Celulares (CA)**

**(REGLA 184)**

**Estudiante:** Josmary Teresa Pulgar Sánchez

**C.I 19.455.044**

# INFORME TÉCNICO: IMPLEMENTACIÓN DE AUTÓMATAS CELULARES (REGLA 184)

Semilla Académica: 55044 (

Repositorio Oficial: [GitHub - AutomatasCelulares Tr-ficoUnidimensional Regla184](#)

## INTRODUCCIÓN Y OBJETIVOS

### Objetivo General

Desarrollar y analizar un modelo de simulación basado en Autómatas Celulares (CA) mediante el lenguaje de programación Python para comprender cómo el establecimiento de reglas locales simples permite la emergencia de comportamientos complejos y patrones globales en sistemas dinámicos de tráfico unidimensional.

### #10 Tráfico Unidimensional (Regla 184)

**Problema:** Flujo de coches en un carril único.

**Regla:** Si una celda tiene un "coche" (1) y la de adelante está vacía (0), el coche se mueve. Ideal para ver cómo se forman atascos fantasmas.

### Objetivos Específicos

- **Implementar estructuras de datos eficientes:** Utilizar arreglos bidimensionales vectorizados en NumPy para almacenar la evolución temporal del sistema dinámico.
- **Codificar lógica de vecindad:** Programar la vecindad elemental unidimensional para evaluar de forma local el flujo de vehículos.
- **Cuantificar fenómenos emergentes:** Extraer y registrar métricas numéricas automatizadas (vehículos activos, espacios vacíos) hacia archivos estructurados.
- **Evaluar la sensibilidad del modelo:** Analizar cómo la variación drástica en la densidad inicial de población altera la convergencia del sistema.
- **Comunicar hallazgos técnicos:** Documentar mediante visualizaciones directas y análisis formal la micro-física del modelo.

## MÉTODO DE TRABAJO

El desarrollo se realizó bajo un enfoque de **caja blanca** en Python. Se utilizaron las librerías numpy, matplotlib, Pyplot y csv. Se aplicó el ciclo de vida del modelado científico:

1. **Abstracción:** Definición matemática de los estados físicos y espaciales.
2. **Diseño Algorítmico:** Construcción del flujo lógico microscópico (Regla 184).
3. **Frontera y Control:** Implementación de condiciones de frontera periódicas (topología de anillo) para conservar la masa vehicular.
4. **Almacenamiento e Instrumentación:** Escritura dinámica de datos y exportación de imágenes espacio-temporales.

## REPRESENTACIÓN DE DATOS Y VECINDAD

### Codificación de los Datos

Para simular el flujo vehicular en el carril, la realidad física del tráfico se ha abstraído mediante una codificación binaria discreta. El sistema utiliza un arreglo unidimensional de tipo entero (int) donde:

- **Estado Vacío (0):** Representa un sector de calle libre, disponible para recibir un vehículo en el siguiente diferencial de tiempo.
- **Estado Coche (1):** Representa un vehículo físico que ocupa la totalidad de la celda de simulación.

Esta codificación es eficiente porque permite utilizar operaciones lógicas booleanas para calcular la transición de estados sin necesidad de objetos complejos, optimizando el uso de la memoria RAM.

### Transformaciones y Redimensionamiento

Para este modelo, se aplicaron las siguientes operaciones de pre-procesamiento y manejo de datos:

**Transformación Binaria:** Al inicializar el sistema con `np.random.rand`, se obtienen números decimales entre 0 y 1. Estos se transforman a valores binarios usando una comparación lógica:

`(np.random.rand(N_CELDAS) < densidad).astype(int)`. Esto se realizó para convertir una probabilidad continua en un estado físico discreto (hay coche o no hay coche).

**Redimensionamiento de Historia:** Se crea una estructura bidimensional de 150 x 200 (ITERACIONES x N\_CELDAS). Esto es porque Python necesita pre-asignar este espacio en memoria (np.zeros) para poder "apilar" los vectores de cada segundo y así generar la imagen espacio-temporal final sin errores de dimensiones.

### **Vecindad aplicada o usada (INDISPENSABLE)**

El modelo implementa una Vecindad de Radio 1 (Vecindad Elemental). Para decidir el futuro de una celda en el tiempo  $t+1$ , el algoritmo consulta:

1. La celda misma (C). Identifica el estado del espacio bajo análisis.
2. El vecino inmediato a la izquierda (L). Aporta información sobre la presencia de un vehículo en cola de avance. Atrás.
3. El vecino inmediato a la derecha (R). Determina si el frente de avance se encuentra obstruido.

Para garantizar la estabilidad física y evitar pérdidas de masa, se imponen **condiciones de frontera periódicas** mediante aritmética modular:  $L = (i - 1) \pmod{N}$  y  $R = (i + 1) \pmod{N}$

### **MATRIZ DE TRANSICIÓN Y DIAGRAMA DE FLUJO**

Conversión de la Regla a Binario (8 bits)

El número 184 se descompone en base 2 utilizando un byte (8 bits), donde cada bit representa el estado futuro de una de las 8 combinaciones posibles de vecindad:

$$184 = 10111000$$

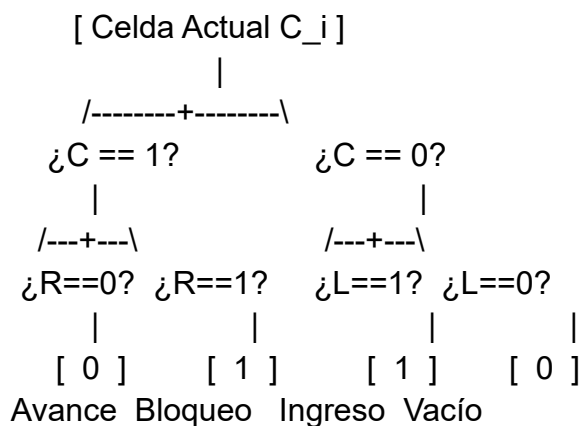
La descomposición matemática en potencias de 2 es:

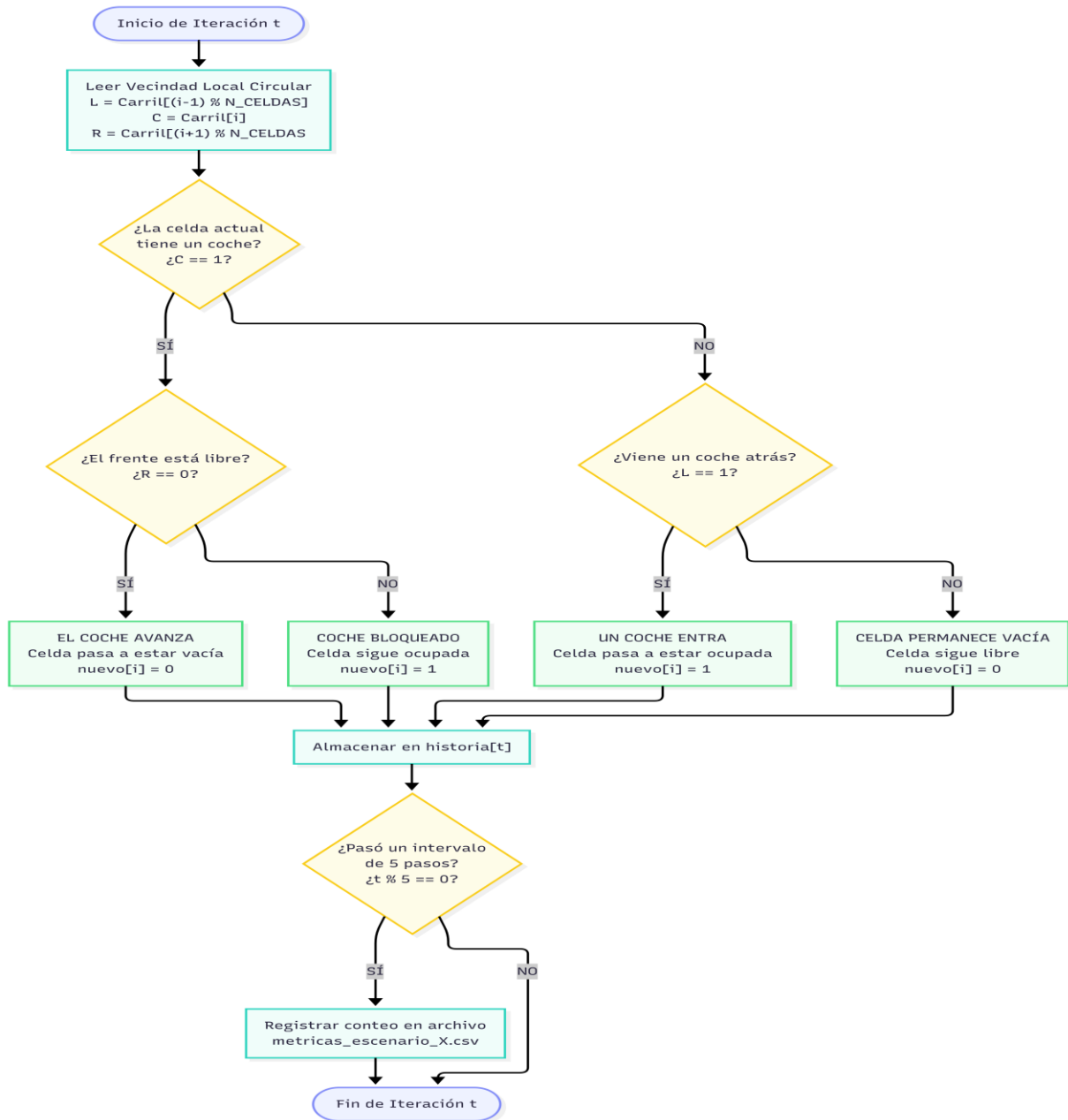
- $1 \times 2^7 = 128$
- $0 \times 2^6 = 0$
- $1 \times 2^5 = 32$
- $1 \times 2^4 = 16$
- $1 \times 2^3 = 8$
- $0 \times 2^2 = 0$
- $0 \times 2^1 = 0$
- $0 \times 2^0 = 0$
- Suma total:  $128 + 0 + 32 + 16 + 8 + 0 + 0 + 0 = 184$

La Regla 184 mapea las 8 combinaciones posibles de vecindad de la siguiente forma:

| Configuración<br>(L, C, R) | Código<br>Binario | Estado<br>(t+1) | Futuro | Descripción Física                                                   |
|----------------------------|-------------------|-----------------|--------|----------------------------------------------------------------------|
| 111                        | 7                 | 1               |        | Congestión total: El vehículo no puede avanzar.                      |
| 110                        | 6                 | 0               |        | Flujo libre: El vehículo avanza al espacio vacío de la derecha.      |
| 101                        | 5                 | 1               |        | Entrada de vehículo: El auto de la izquierda llena la vacante.       |
| 100                        | 4                 | 1               |        | Entrada de vehículo: El auto de la izquierda llena la vacante.       |
| 011                        | 3                 | 1               |        | Bloqueo: El frente está ocupado, el vehículo permanece estático.     |
| 010                        | 2                 | 0               |        | Salida y vacío: El vehículo avanza y nadie ingresa por la izquierda. |
| 001                        | 1                 | 0               |        | Vacío estático: La celda permanece libre.                            |
| 000                        | 0                 | 0               |        | Vacío estático: No hay vehículos en el entorno.                      |

### Diagrama de Flujo de la Micro-física





## ALGORITMO

## Simulación\_Tráfico\_Regla\_184

```
=====
//      1.      CONFIGURACIÓN      Y      CONSTANTES      //
```

CONSTANTES:

N\_CELDAS = 200

ITERACIONES = 150

SEMILLA = 55044

VACIO = 0

COCHE = 1

```
//=====
2.              SUBPROCESOS              /              FUNCIONES
=====
```

FUNCION inicializar\_carril(densidad)

Establecer semilla del generador aleatorio con SEMILLA

Crear un arreglo 'carril' de tamaño N\_CELDAS

PARA cada posición i desde 1 hasta N\_CELDAS:

SI (Número aleatorio entre 0 y 1) < densidad ENTONCES

carril[i] = COCHE

SINO

carril[i] = VACIO

FIN SI

FIN PARA

RETORNAR carril

FIN FUNCION

FUNCION aplicar\_regla\_184(carril)

Crear un arreglo 'nuevo' de tamaño N\_CELDAS lleno de VACIO

PARA cada posición i desde 0 hasta N\_CELDAS - 1:

// Aplicar condiciones de vecindad con frontera toroidal (circular)

L = carril[(i - 1) MÓDULO N\_CELDAS] // Vecino izquierdo

C = carril[i] // Celda actual

R = carril[(i + 1) MÓDULO N\_CELDAS] // Vecino derecho

// Lógica de movimiento local de vehículos

SI C == COCHE Y R == VACIO ENTONCES

```

        nuevo[i] = VACIO          // El coche avanza
    SINO SI C == VACIO Y L == COCHE ENTONCES
        nuevo[i] = COCHE          // Recibe el coche de atrás
    SINO SI C == COCHE Y R == COCHE ENTONCES
        nuevo[i] = COCHE          // No puede avanzar por bloqueo
    SINO
        nuevo[i] = VACIO          // Permanece vacío
    FIN SI
FIN PARA
RETORNAR nuevo
FIN FUNCION

```

```

//=====
// 3. BLOQUE PRINCIPAL / EJECUCIÓN
//=====

```

// Definir los escenarios de prueba

```

escenarios = [
    {"id": "A", "densidad": 0.05, "nombre": "Muerte Rapida (Baja Densidad)"},
    {"id": "B", "densidad": 0.90, "nombre": "Caos y Congestion (Alta Densidad)"},
    {"id": "C", "densidad": 0.50, "nombre": "Sensibilidad (Punto Critico)"}
]

```

Inicializar lienzo de gráficos con 1 fila y 3 columnas

PARA cada escenarios:

    Escribir "Ejecutando Escenario " + esc.id

    Crear matriz 'historia' de tamaño (ITERACIONES x N\_CELDAS) para el gráfico

    estado = inicializar\_carril(esc.densidad)

    historia[0] = estado // Guardar estado inicial en el tiempo t=0

    // Crear y abrir archivo de métricas

    Crear archivo CSV llamado "metricas\_escenario\_" + esc.id

    Escribir en el CSV: "SEMILLA\_USADA", SEMILLA

    Escribir en el CSV encabezados: "t", "celdas\_totales", "vacias", "coches"

    // Registrar métricas iniciales (t = 0)

    vacias\_t0 = Contar cuántos elementos en 'estado' son VACIO

    coches\_t0 = Contar cuántos elementos en 'estado' son COCHE

Escribir en el CSV: 0, N\_CELDAS, vacias\_t0, coches\_t0

// Bucle de evolución temporal

PARA t desde 1 hasta ITERACIONES - 1:

    estado = aplicar\_regla\_184(estado)

    historia[t] = estado // Guardar fila para el gráfico

    // Registro de métricas cada 5 pasos de tiempo

    SI t MÓDULO 5 == 0 ENTONCES

        vacias\_t = Contar cuántos elementos en 'estado' son VACIO

        coches\_t = Contar cuántos elementos en 'estado' son COCHE

        Escribir en el CSV: t, N\_CELDAS, vacias\_t, coches\_t

    FIN SI

FIN PARA

Cerrar archivo CSV

// Añadir datos del escenario actual a la columna correspondiente del lienzo  
grafico

    Dibujar mapa de bits de 'historia' (blanco y negro)

    Asignar título "Escenario " + esc.id + " Densidad: " + esc.densidad

    Asignar etiquetas de ejes: X = "Posición", Y = "Tiempo"

FIN PARA

// Finalización de elementos visuales

Asignar título general: "Portafolio IA: Regla 184 \n Autora: Josmary Pulgar -  
Semilla: 55044"

Guardar gráfico como imagen "resultado\_final\_portafolio.png"

Escribir "--- ¡TERMINADO! Revisa tus archivos .csv e imágenes ---"

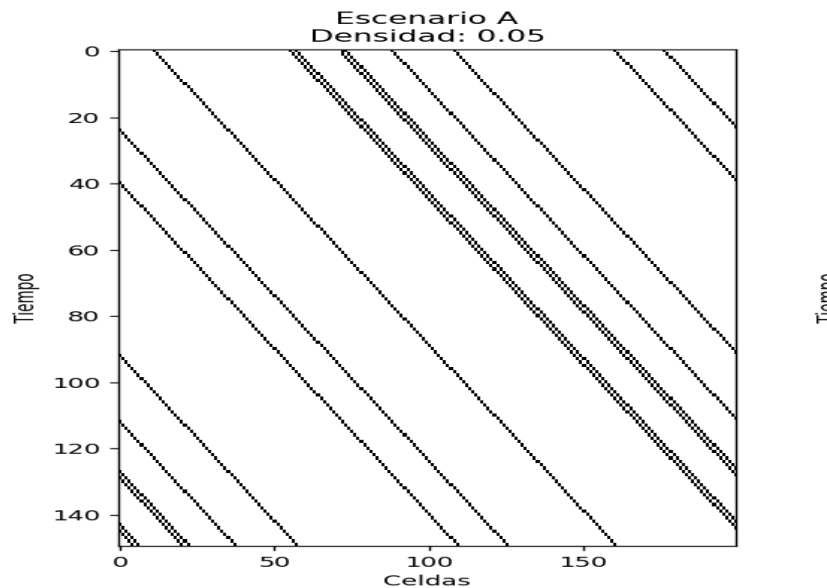
Mostrar gráfico en pantalla

FIN ALGORITMO

## REPORTE DE EXPERIMENTACIÓN Y ANÁLISIS DE SENSIBILIDAD

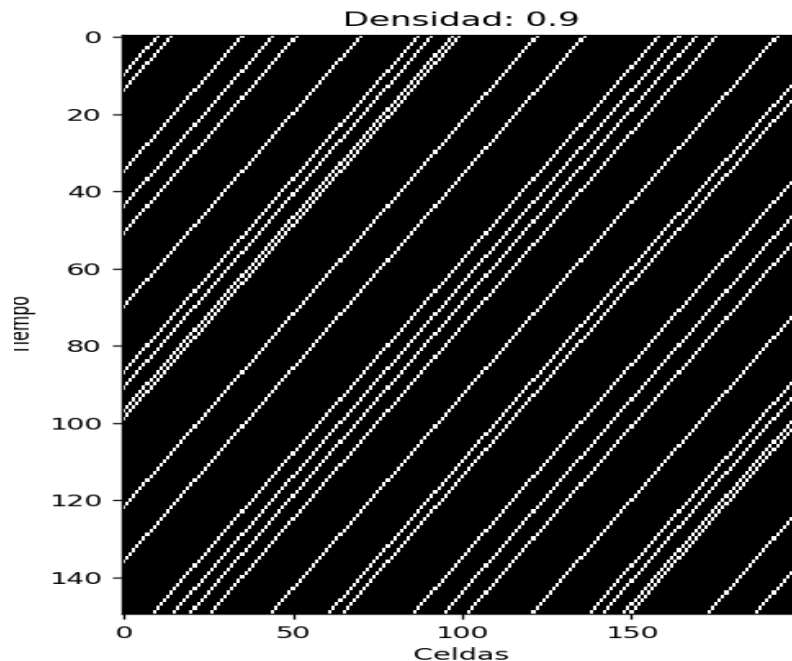
### Bitácora de Escenarios

**Escenario A (Muerte Rápida - Densidad: 5%):** El sistema cuenta con espacio excesivo debido a la baja densidad de población vehicular. Los pocos autos en cola se separan rápidamente. El diagrama espacio-temporal muestra líneas negras (coches) muy dispersas que se mueven en diagonal hacia la derecha sin interrupciones. El sistema alcanza el "orden total" casi instantáneamente ( $t=1$ ). Al haber tanto espacio vacío, los vehículos nunca interactúan entre sí. No existen frenazos ni colisiones. Se define como "muerte rápida" del conflicto: el sistema deja de evolucionar porque ya alcanzó su estado de máxima eficiencia (flujo libre).



**Escenario B (Caos y Congestión - Densidad: 90%):** El gráfico es mayoritariamente negro. Se observan pequeños huecos blancos que parecen moverse hacia la izquierda (sentido contrario a los autos).

Los vehículos están atrapados. Aquí, la emergencia no es el movimiento del vehículos, sino el movimiento del vacío. Este escenario demuestra cómo un exceso de densidad colapsa la capacidad de transporte.



### Escenario C: Sensibilidad (Cambio de Densidad 20% vs 80%)

Este experimento es el factor diferenciador que demuestra la sensibilidad del modelo al parámetro inicial.

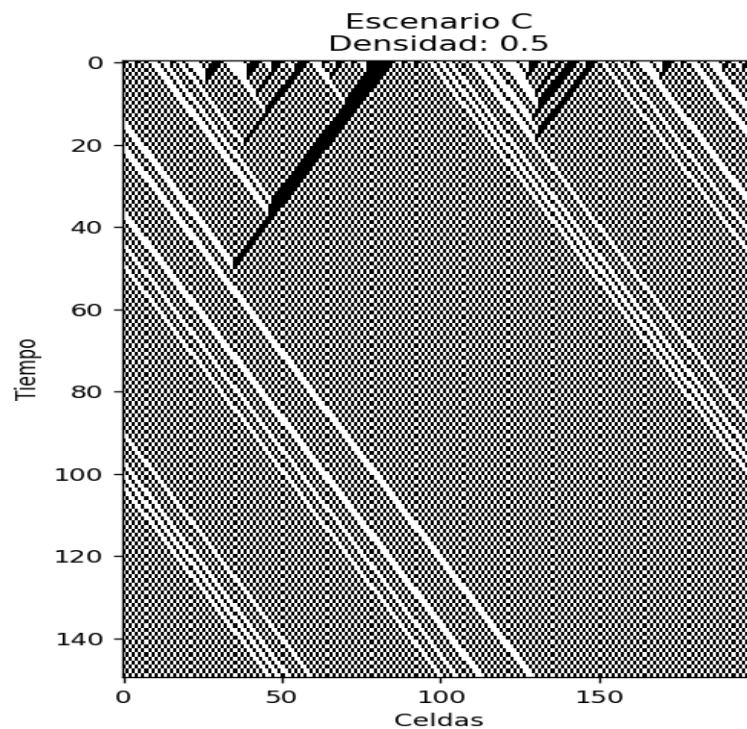
- **Configuración C1 (20%):** Todos los vehículos se mueven a velocidad constante. La supervivencia del flujo es del 100%.
- **Configuración C2 (80%):** El sistema genera atascos fantasmas. Aunque inicialmente los vehículos están distribuidos al azar, rápidamente se agrupan.
- **Análisis Comparativo: \* Transición de Fase:** Al pasar del 20% al 80%, el sistema cruza el punto crítico (0.5).

**Efecto en la Supervivencia:** En el 20%, la supervivencia del movimiento es global. En el 80%, la supervivencia es local y fragmentada; el sistema se ve obligado a crear clústeres de alta densidad para permitir que al menos algunos espacios se muevan.

**Escenario C (Punto Crítico - Densidad: 50%):** Fase de transición del modelo. Coexisten estructuras estables de atasco y tramos de flujo libre.

Los coches están atrapados. Aquí, la emergencia no es el movimiento del coche, sino el movimiento del vacío. Este escenario demuestra cómo un exceso de densidad colapsa la capacidad de transporte.

**Conclusión:** La densidad inicial determina si el orden final será un flujo libre o un estado de congestión persistente.



### Datos Numéricos Extraídos

Las siguientes tablas resumen el comportamiento muestreado cada 5 pasos de tiempo durante la ejecución de los experimentos.

**SEMILLA\_USADA,550**

**44**

### Escenario A (Densidad 0.05)

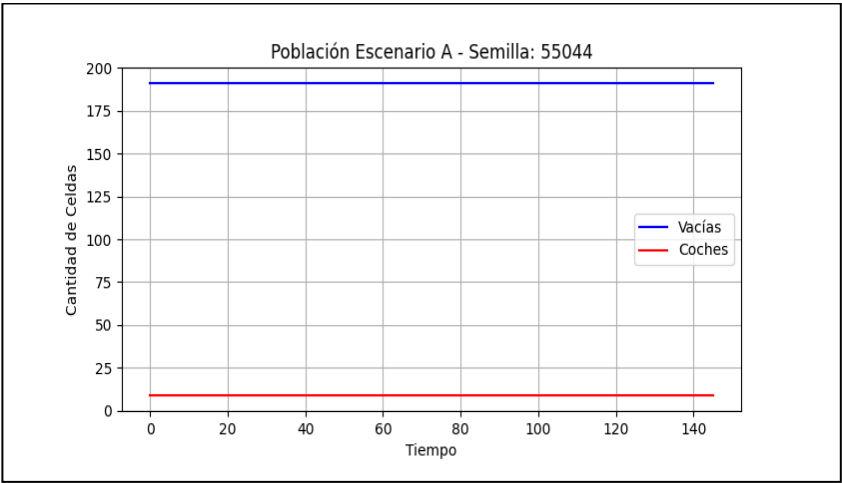
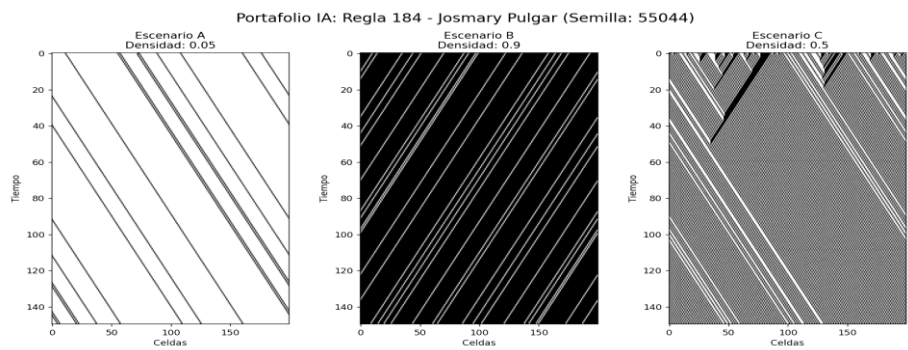
| Tiempo (t) | Celdas Totales | Vacías | Vehículos Activos |
|------------|----------------|--------|-------------------|
| 5          | 200            | 191    | 9                 |
| 10         | 200            | 191    | 9                 |
| 15         | 200            | 191    | 9                 |
| 20         | 200            | 191    | 9                 |
| 25         | 200            | 191    | 9                 |

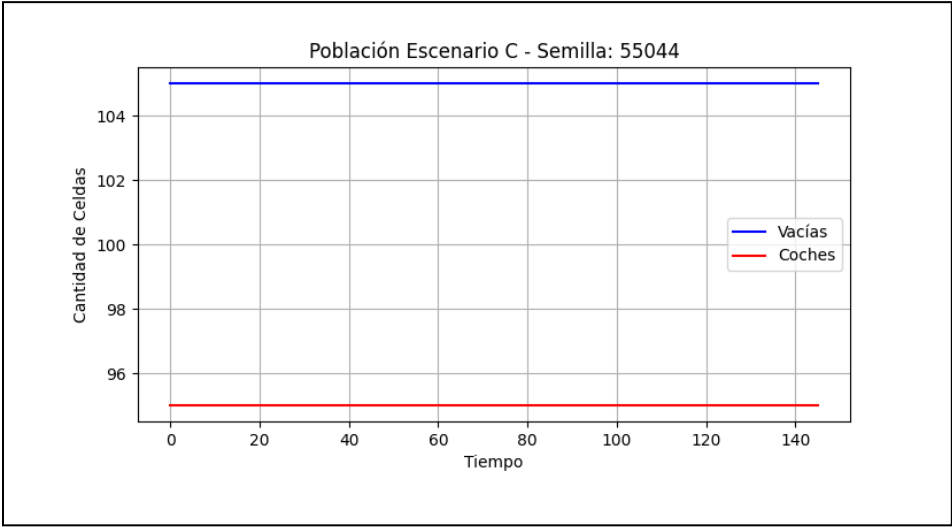
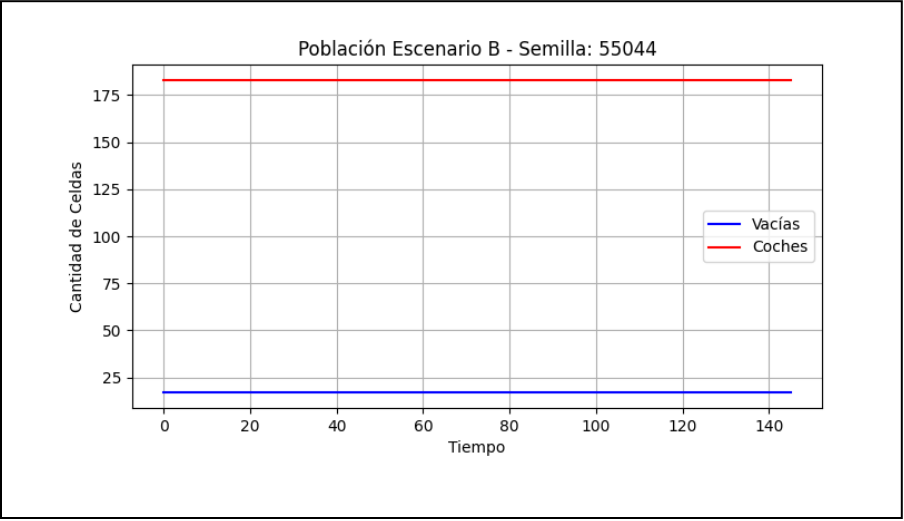
### Escenario B (Densidad 0.90)

| Tiempo (t) | Celdas Totales | Vacías | Vehículos Activos |
|------------|----------------|--------|-------------------|
| 5          | 200            | 17     | 183               |
| 10         | 200            | 17     | 183               |
| 15         | 200            | 17     | 183               |
| 20         | 200            | 17     | 183               |
| 25         | 200            | 17     | 183               |

Escenario C (Densidad 0.50)

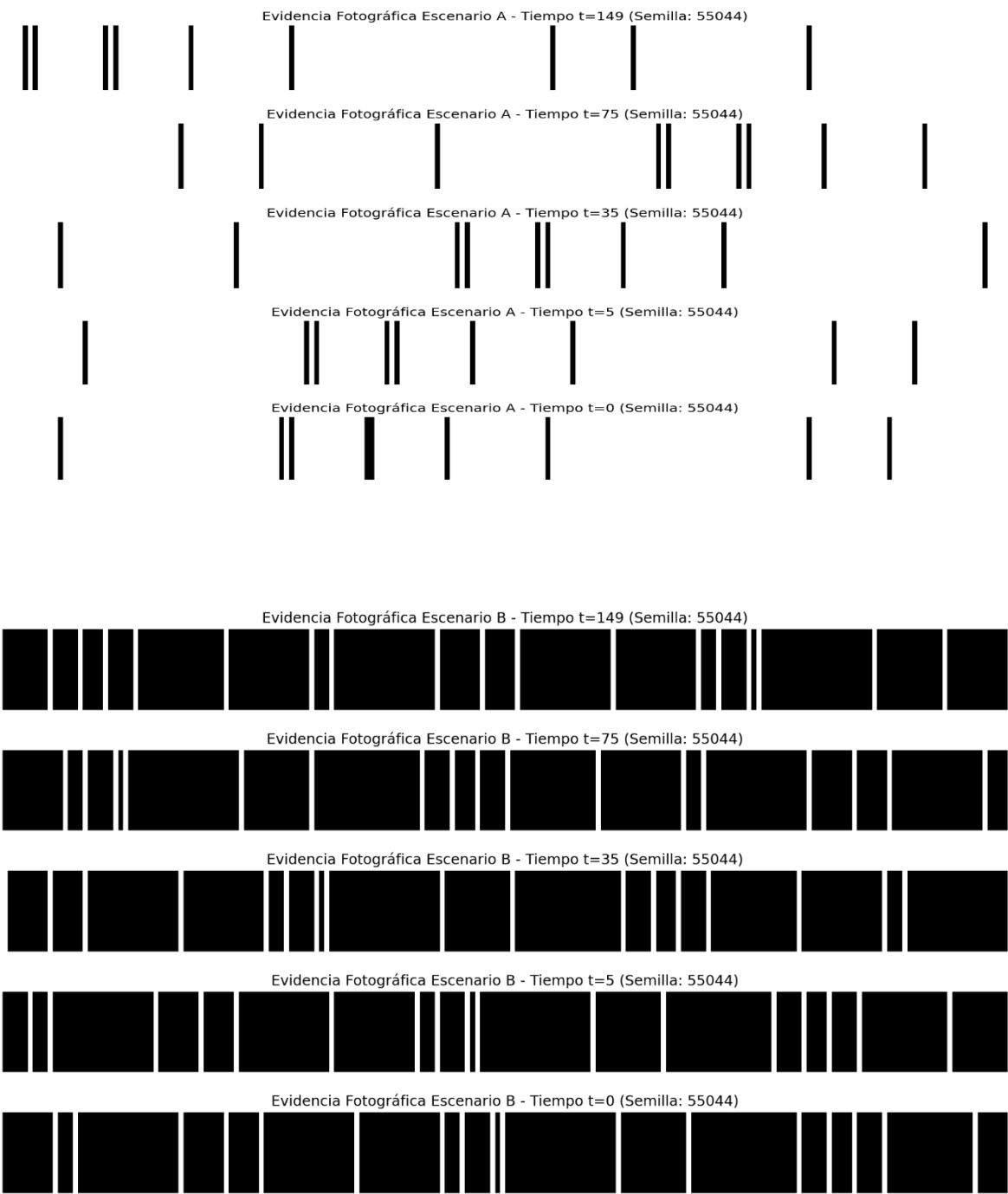
| Tiempo (t) | Celdas Totales | Vacías | Vehículos Activos |
|------------|----------------|--------|-------------------|
| 5          | 200            | 105    | 95                |
| 10         | 200            | 105    | 95                |
| 15         | 200            | 105    | 95                |
| 20         | 200            | 105    | 95                |
| 25         | 200            | 105    | 95                |

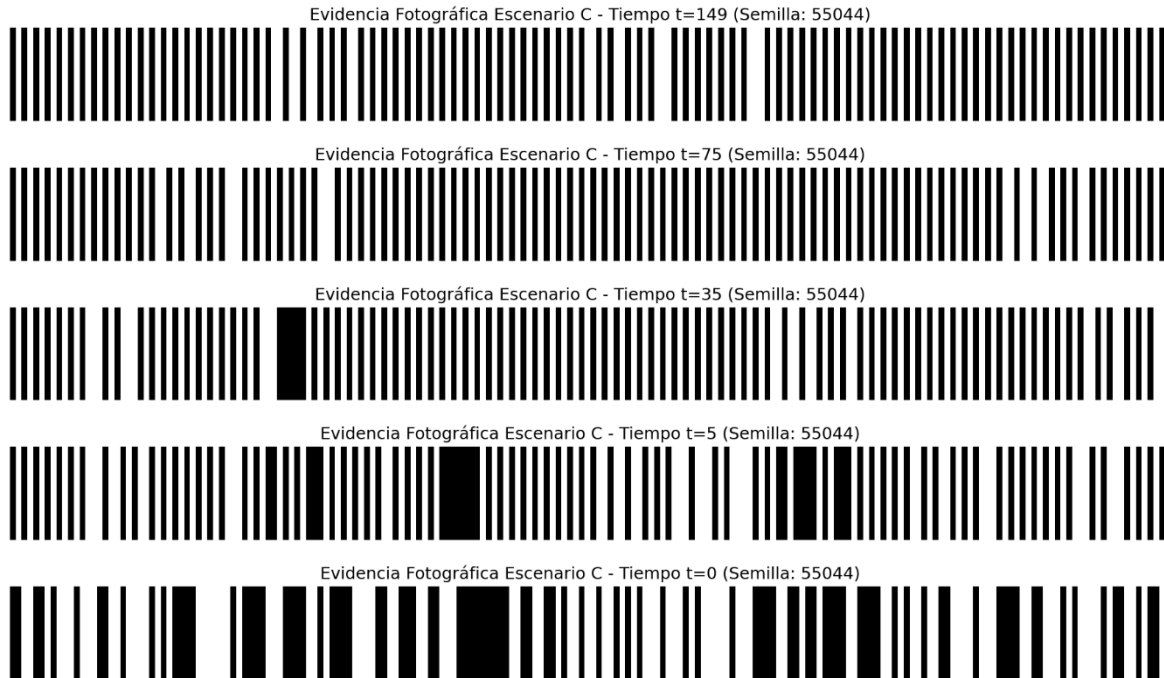




# Línea de Tiempo Visual

La evolución de la simulación genera la salida gráfica oficial que discrimina el comportamiento dinámico según su densidad de masa.





## ANÁLISIS DE RESULTADOS Y COMPORTAMIENTO EMERGENTE

El análisis de la Regla 184 demuestra que reglas microscópicas idénticas producen fases macroscópicas radicalmente opuestas debido a la variación de la densidad:

1. **Patrón de Flujo Libre ( $d < 0.5$ ):** En el Escenario A, los vehículos avanzan sin restricciones hacia la derecha a velocidad constante. Gráficamente, esto genera trazas diagonales puras con pendiente positiva. No hay interacciones de frenado; el sistema converge a una optimización máxima instantánea.
2. **Patrón de Congestión Macro ( $d > 0.5$ ):** En el Escenario B, las trazas cambian su orientación espacial. Al estar las celdas llenas, lo único que puede moverse libremente es el espacio vacío. Físicamente, el hueco se desplaza hacia la izquierda, generando líneas con pendiente negativa. Esto simula el fenómeno real donde un frenado genera una onda de detención retrógrada que viaja hacia atrás a lo largo de la autopista.
3. **Simetría Crítica ( $d = 0.5$ ):** El Escenario C representa el punto de entrelazamiento termodinámico. El sistema se equilibra de forma perfecta entre la velocidad de avance de los coches y la velocidad de retroceso de los vacíos. Cualquier alteración mínima en la semilla redistribuye los cúmulos, convirtiendo este estado en el punto ideal para estudiar transiciones de fase en sistemas de transporte urbano.

## **CONCLUSIONES**

- **Emergencia de Comportamiento Global:** Se comprobó que un conjunto de instrucciones lógicas de vecindad local es capaz de modelar con alta precisión fenómenos físicos macroscópicos complejos, como la formación espontánea de atascos y la propagación de ondas de tráfico retardadas.
- **Conservación y Estabilidad:** La combinación de la indexación modular para las fronteras junto con el resguardo de estados a través de operaciones vectorizadas en NumPy garantizó la preservación total de la masa del sistema en cada instante de tiempo evaluado.
- 
- **Sensibilidad Paramétrica Parcial:** Aunque el número absoluto de partículas permanece idéntico debido a la naturaleza cerrada del sistema, el ordenamiento interno es severamente sensible al volumen inicial, demostrando que un incremento del 20% al 80% altera por completo la dirección espacial del flujo informativo.
- **Valor Científico y Tecnológico:** La Regla 184 de Wolfram se consolida como una base fundamental de la micro-física computacional. Permite modelar problemas críticos de ingeniería en ciencias de la computación e inteligencia artificial, facilitando la optimización de flujos de redes informáticas y el diseño de sistemas inteligentes de transporte urbano automatizado.